

# Financial Risk Analysis for Loan Approval

AI Engineer Role | Pre-Work Assignment | Option 1

Karishma Patil | ML Engineer | Bangalore | pkarishma8989@gmail.com

## Section 1: Context

### Project Description

This project involved building an end-to-end machine learning pipeline to assess credit risk and automate loan approval decisions for a banking application. The system predicts two outputs: (1) whether a loan application should be approved (classification), and (2) the estimated loan amount to disburse (regression). The solution was deployed via a Flask REST API on AWS EC2, allowing the bank's internal systems to get real-time predictions on customer applications.

### Primary Technical Constraints

| Constraint       | Details                                                                                 |
|------------------|-----------------------------------------------------------------------------------------|
| Class Imbalance  | Approved vs. rejected loans were heavily skewed (~80:20), requiring SMOTE oversampling  |
| Missing Data     | ~15-20% of customer records had missing income, credit history, and employment fields   |
| Latency          | API response time needed to stay under 500ms for real-time approval decisions           |
| Interpretability | Banking domain requires explainable predictions — black-box models were discouraged     |
| Data Privacy     | Customer PII had to stay within AWS infrastructure; no external API calls with raw data |

## Section 2: Technical Implementation

### Architecture Overview

Raw customer data enters through a Flask API endpoint, passes through a shared preprocessing pipeline (KNN imputation → outlier treatment → feature scaling), and feeds into two separate Random Forest models — one for approval classification and one for loan amount regression. Prediction results are returned as a structured JSON response, and all prediction logs are stored in AWS S3 for audit and monitoring.

### Code Walk-through: Critical Function — preprocess and predict()

Every API call flows through this single function. It handles imputation, outlier treatment, scaling, and dual-model inference in one pass. Any bug here directly impacts production loan decisions, which is why this is the most critical function in the system.

```
def preprocess_and_predict(raw_input: dict) -> dict:
    df = pd.DataFrame([raw_input])
```

```

df = encode_categoricals(df)

# Step 1: Impute missing values using KNN (k=5)
df_imputed = pd.DataFrame(knn_imputer.transform(df), columns=df.columns)

# Step 2: Clip outliers using IQR bounds from training set
for col in NUMERIC_COLS:
    df_imputed[col] = df_imputed[col].clip(
        lower=iqr_bounds[col]["lower"],
        upper=iqr_bounds[col]["upper"])

# Step 3: Scale features
df_scaled = scaler.transform(df_imputed)

# Step 4: Classification – approved or rejected?
approval_proba = clf_model.predict_proba(df_scaled)[0][1]
approval_decision = int(approval_proba >= APPROVAL_THRESHOLD)

# Step 5: Estimate loan amount only if approved
loan_amount = None
if approval_decision == 1:
    loan_amount = float(reg_model.predict(df_scaled)[0])

return {
    "approved": bool(approval_decision),
    "approval_probability": round(float(approval_proba), 4),
    "estimated_loan_amount": round(loan_amount, 2) if loan_amount else None
}

```

### **Data Flow for Loan Approval Prediction**

Raw JSON input is received by the Flask API → required fields are validated → KNN imputer fills missing numeric values using 5 nearest-neighbor profiles → IQR clipping caps outliers at training-set bounds → StandardScaler normalizes features → Random Forest Classifier generates an approval probability score → if probability  $\geq 0.5$ , the application is marked Approved and the loan amount regressor estimates the disbursement amount → a structured JSON response is returned to the calling system.

## **Section 3: Technical Decisions**

---

### **Decision 1: Random Forest over Logistic Regression**

|                       | Random Forest               | Logistic Regression |
|-----------------------|-----------------------------|---------------------|
| Accuracy              | 85%                         | 79%                 |
| Handles Non-linearity | Yes                         | No                  |
| Interpretability      | Medium (feature importance) | High (coefficients) |
| Training Speed        | Slower                      | Fast                |
| Overfitting Risk      | Low (ensemble)              | Low                 |

**Why I chose Random Forest:** The customer dataset had non-linear relationships between features — for example, interactions between income and credit score that Logistic Regression could not capture. Random

Forest gave 85% accuracy compared to 79% with Logistic Regression. Feature importance scores also gave partial explainability to banking stakeholders.

**Trade-off accepted:** Random Forest is slower to train and harder to interpret. I mitigated inference latency by pre-loading the serialized model using joblib at Flask startup, keeping API response time under 100ms.

### **Decision 2: KNN Imputation over Mean/Median Imputation**

|                                 | <b>KNN Imputation</b> | <b>Mean/Median Imputation</b> |
|---------------------------------|-----------------------|-------------------------------|
| Preserves Feature Relationships | Yes                   | No                            |
| Handles Non-random Missingness  | Better                | Weak                          |
| Computation Cost                | Higher                | Very Low                      |
| Impact on Model Accuracy        | +2-3% improvement     | Baseline                      |

**Why I chose KNN Imputation:** Customer data had Missing Not At Random (MNAR) patterns — self-employed customers were more likely to leave income fields blank. Mean imputation would have introduced systematic bias. KNN imputer fills values based on 5 similar customer profiles, preserving feature correlations and improving accuracy by 2-3%.

**Trade-off accepted:** KNN imputation is computationally heavier. During batch retraining on 50,000+ records it added ~3 extra minutes. This was acceptable given the accuracy gain.

### **Scaling Bottleneck and Mitigation Strategy**

**Bottleneck identified:** The Flask API was single-threaded and synchronous. Under concurrent submissions (100+ requests/second during peak hours), response times spiked from 80ms to over 4 seconds.

Steps taken to mitigate:

- Switched from Flask development server to Gunicorn with 4 async workers on EC2
- Pre-loaded all model artifacts (imputer, scaler, classifier, regressor) into memory at startup using `joblib.load()` — not per request
- Configured AWS EC2 Auto Scaling group to add instances when CPU utilization exceeded 70%
- Future plan: Migrate to AWS SageMaker endpoints for native autoscaling and monitoring

## **Section 4: Learning & Iteration**

---

### **One Technical Mistake and What I Learned**

**Mistake:** I applied SMOTE oversampling to the entire dataset before splitting into train and test sets. This caused data leakage — synthetic samples from the training distribution appeared in the test set, artificially inflating accuracy from ~82% to ~89%.

**What I learned:** Always perform the train-test split first, then apply oversampling only on the training set. The test set must represent real, unseen data and should never be augmented or synthetic. I now use Scikit-learn Pipelines to enforce the correct ordering and prevent this mistake in future projects.

### **One Thing I Would Do Differently Today**

I would implement model monitoring and drift detection from day one of deployment. After the model went live, its performance gradually degraded as customer loan patterns shifted due to economic changes. We had no automated alerts in place, and the issue was only caught weeks later when business stakeholders flagged unusual approval rates.

Today, I would integrate Evidently AI or AWS Model Monitor to continuously track feature distribution drift (using PSI score), prediction distribution shifts, and rolling model accuracy. Any significant drift would trigger an automatic alert to the ML team within 24 hours, enabling proactive retraining before business impact occurs.

### **Results Summary**

---

| Metric                            | Score       |
|-----------------------------------|-------------|
| Classification Accuracy           | 85%         |
| AUC-ROC Score                     | 0.91        |
| Regression R2 Score               | 0.82        |
| Mean Absolute Error (Loan Amount) | ~12,000 INR |
| API Response Time                 | < 100ms     |

*Tech Stack: Python | Scikit-learn | Pandas | NumPy | Flask | AWS EC2 | AWS S3 | joblib | SMOTE | GridSearchCV | Random Forest*

*Thank you for reviewing my submission. I am happy to walk through any section in more depth during the next round.*